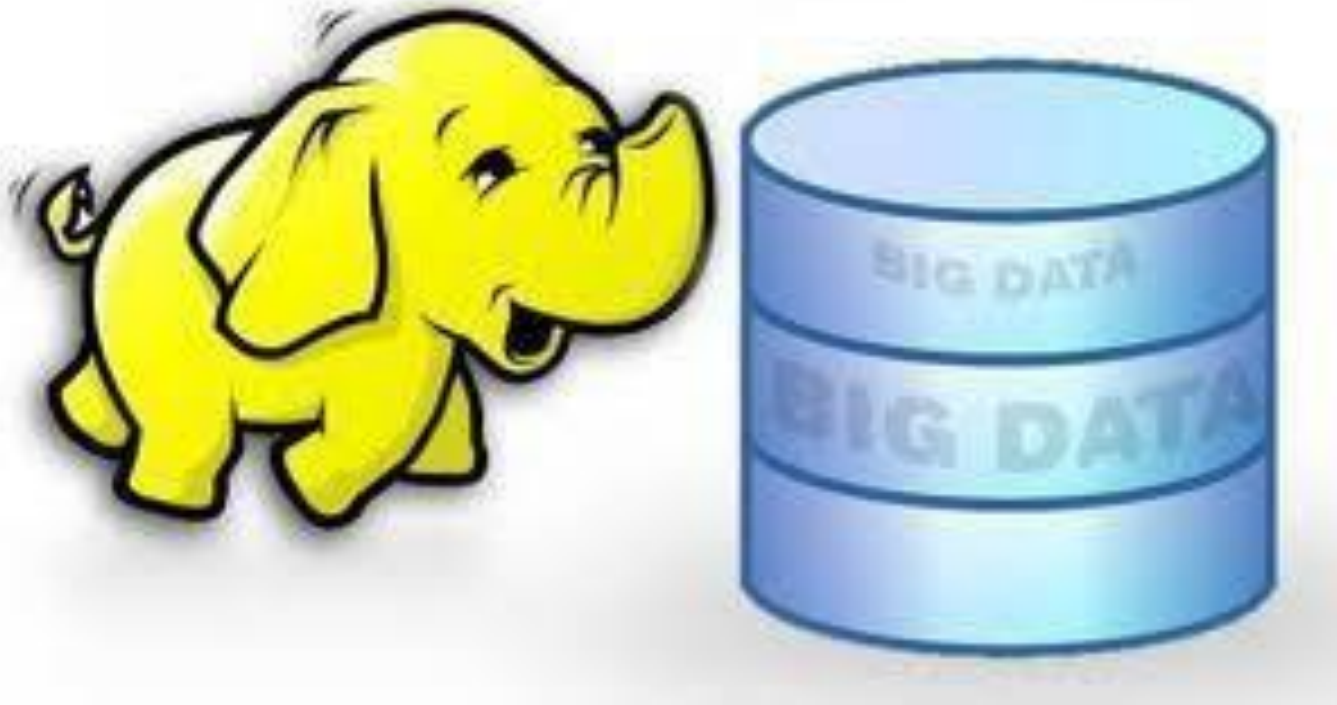


Chapitre 2

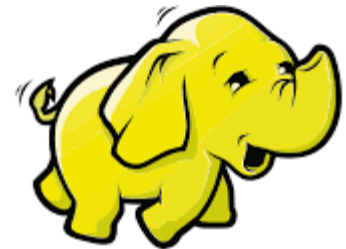
Hadoop



Hadoop

Hadoop (**H**igh-**A**vailability **D**istributed **O**bject-**O**riented **P**latform) :

- Framework libre et open source
- Ecrit en Java
- Géré par Apache
- Capable de s'attaquer à un gros volume de données de types différents



hadoop

Hadoop: Présentation (1/3)

- **Hadoop** est une plateforme (framework) open source conçue pour réaliser d'une façon distribuée des traitements sur des volumes de données massives, de l'ordre de plusieurs pétaoctets. Ainsi, il est destiné à faciliter la création d'applications distribuées et échelonnables (**scalables**). Il s'inscrit donc typiquement sur le terrain du Big Data.
- **Hadoop** est géré sous l'égide de la fondation **Apache**, il est écrit en Java.
- **Hadoop** a été conçu par Doug Cutting en 2004 et été inspiré par les publications MapReduce, GoogleFS et BigTable de Google.
-

Hadoop: Présentation (2/3)

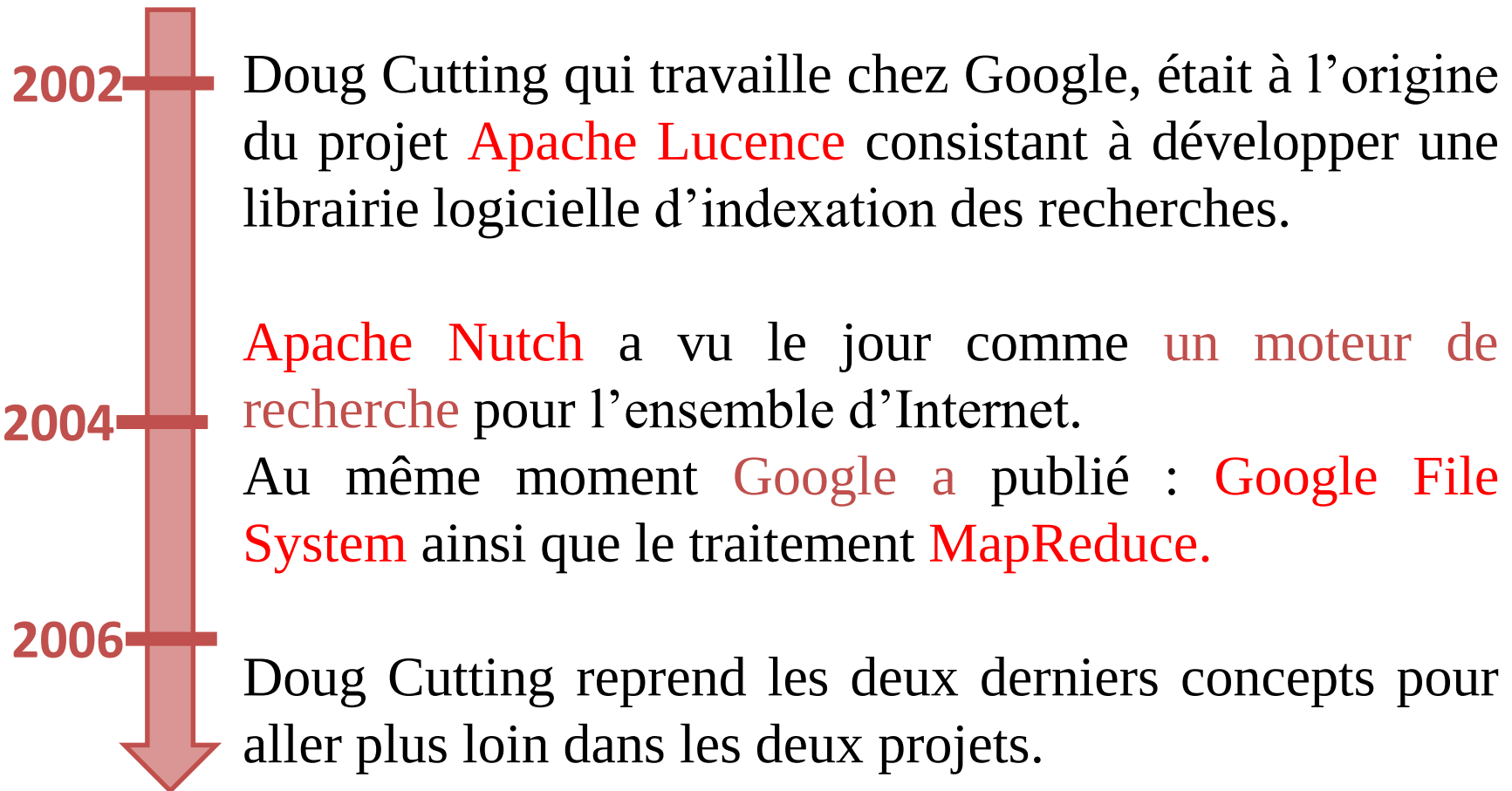
- Modèle simple pour les développeurs: il suffit de développer des tâches Map-Reduce, depuis des interfaces simples accessibles via des librairies dans des langages multiples (Java, Python, C/C++...).
- Déployable très facilement (paquets Linux pré-configurés), configuration très simple elle aussi.
- S'occupe de toutes les problématiques liées au calcul distribué, comme l'accès et le partage des données, la tolérance aux pannes, ou encore la répartition des tâches aux machines membres du cluster : le programmeur a simplement à s'occuper du développement logiciel pour l'exécution de la tâche.

Hadoop: Présentation (3/3)

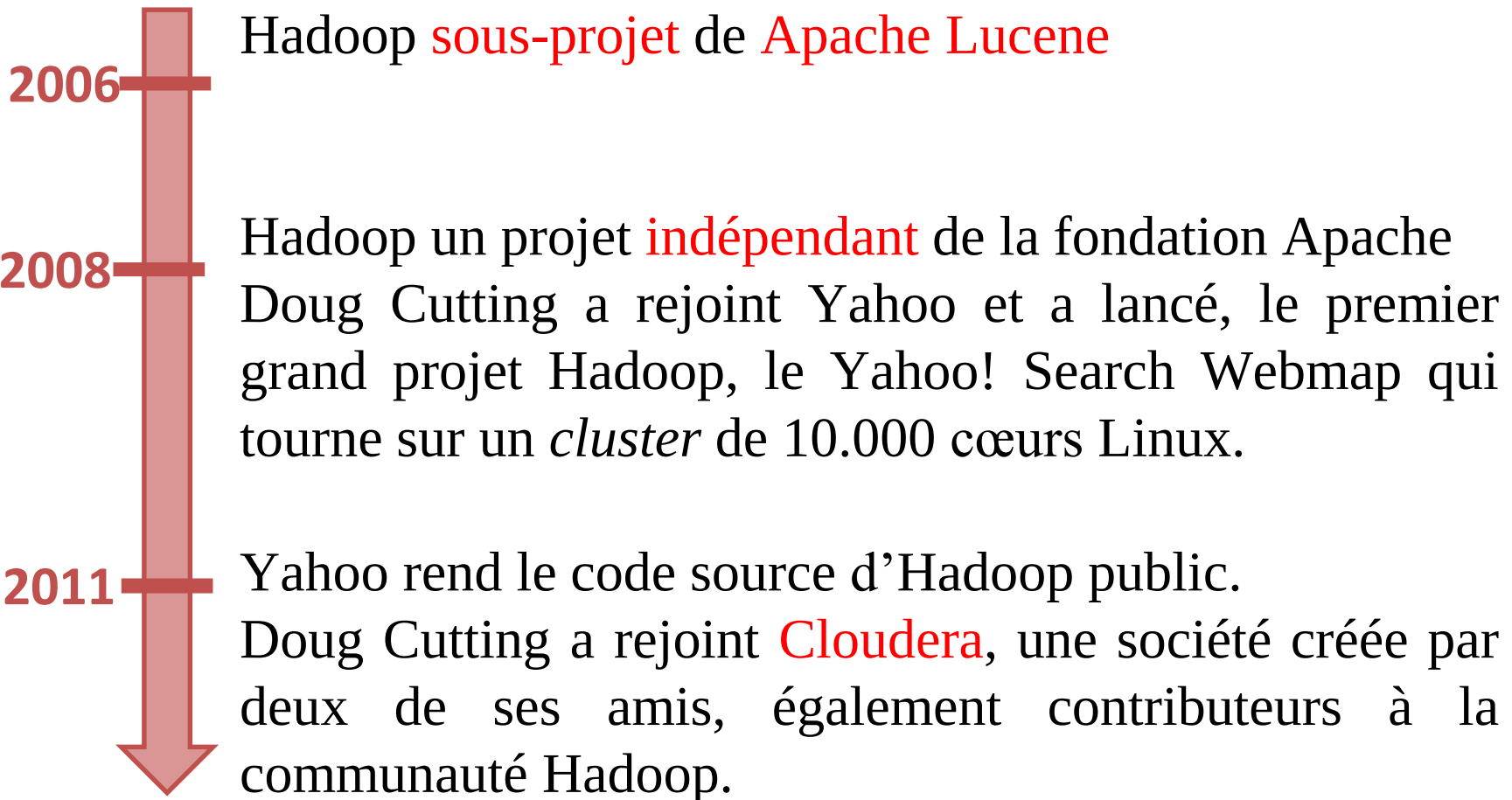
Hadoop assure les critères des solutions Big Data:

- **Performance:** support du traitement d'énormes data sets (millions de fichiers, Go à To de données totales) en exploitant le parallélisme et la mémoire au sein des clusters de calcul.
- **Economie:** contrôle des coûts en utilisant de matériels de calcul de type standard.
- **Evolutivité (scalabilité):** un plus grand cluster devrait donner une meilleure performance.
- **Tolérance aux pannes:** la défaillance d'un nœud ne provoque pas l'échec de calcul.
- **Parallélisme de données:** le même calcul effectué sur toutes les données.

Histoire de Hadoop



Histoire de Hadoop



Hadoop

Plusieurs entreprises utilisent Hadoop :

- Amazon
- Adobe
- Facebook
- Google
- Ebay
- Twitter
- Yahoo

Liste :

<https://wiki.apache.org/hadoop/PoweredBy>

Hadoop

Hadoop fournit :

- Système distribué qui répond aux nouvelles dimensions des données.
- Système de fichiers **HDFS** (Hadoop Distributed File System) permettant de stocker la donnée en la dupliquant.
- Système de traitement des gros volumes de données appelé **MapReduce**.

Hadoop

Principe :

- Diviser les données
- Les sauvegarder sur une collection de machines, appelées cluster
- Traiter les données directement là où elles sont stockées, plutôt que de les copier à partir d'un serveur distribué

Il est possible d'ajouter des machines à un cluster, au fur et à mesure que les données augmentent.

Distributions de Hadoop

Trois principales distributions Hadoop :

- Cloudera
- Hortonworks
- MapR
- AWS (RosettaHub)

Ecosystème de Hadoop

En plus des briques de base [Yarn/MapReduce/HDFS](#), plusieurs outils existent pour permettre :

- L'extraction et le stockage des données de/sur HDFS
- La simplification des traitements sur ces données
- La gestion et la coordination de la plateforme
- Le monitoring du cluster

Hadoop : Choix



Apache Hadoop



Outils & Support commercial avec :
outils graphiques pour le déploiement,
l'administration et le monitoring des clusters

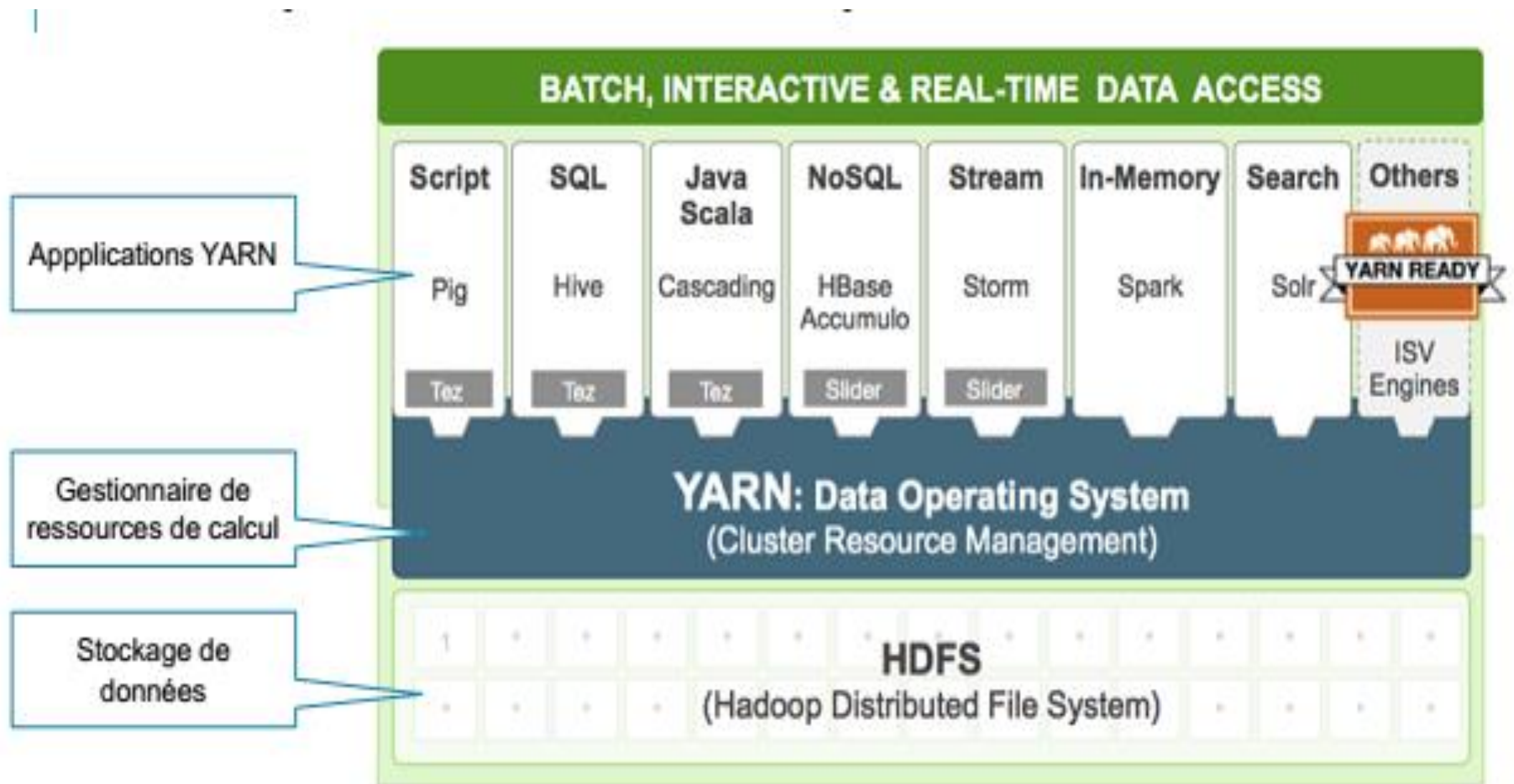
Hadoop Distribution



Plug ins / connecteurs additionnels
+
Modélisation & Configurateur graphique des jobs MR +
Planificateur

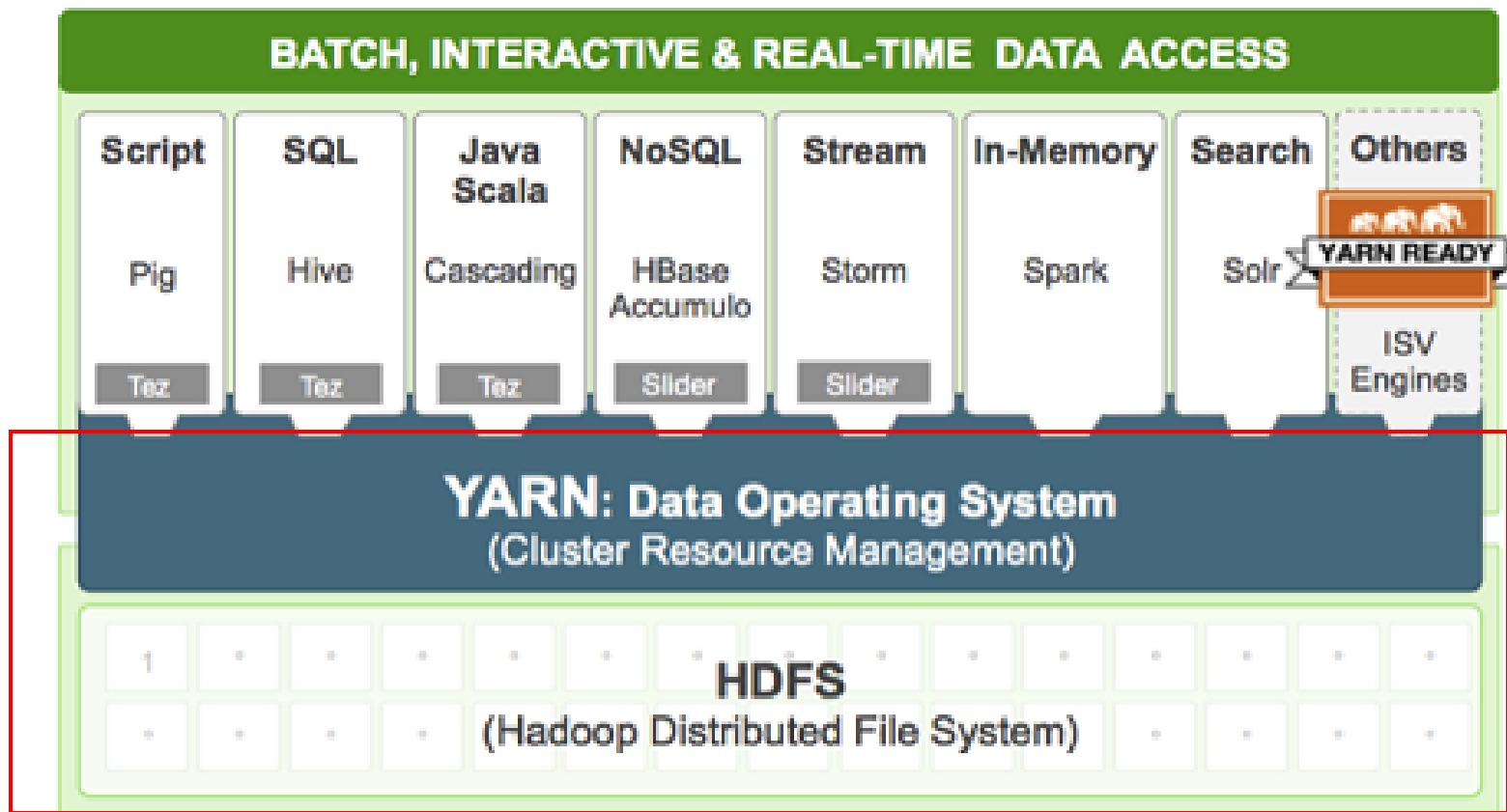
Big Data Suite

Ecosystème de Hadoop



Echystème de Hadoop

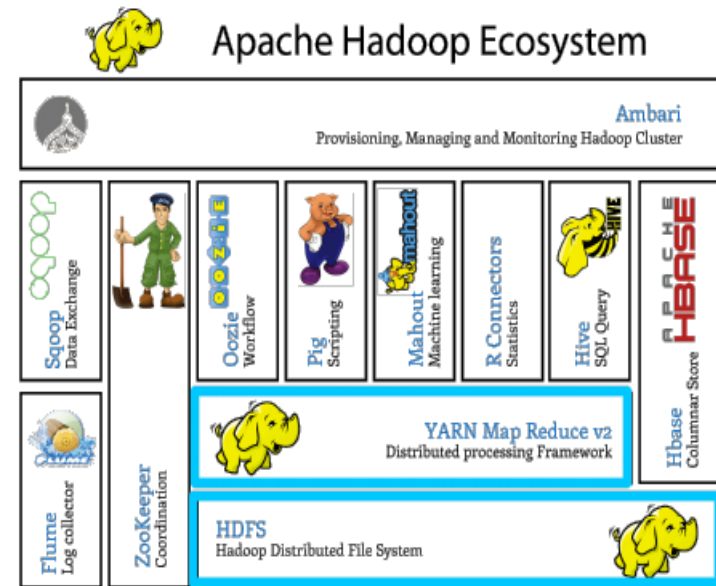
Système d'exploitation



Ecosystème de Hadoop

Certains de ces outils se trouvent **au dessus de la couche Yarn/MR**, tels que :

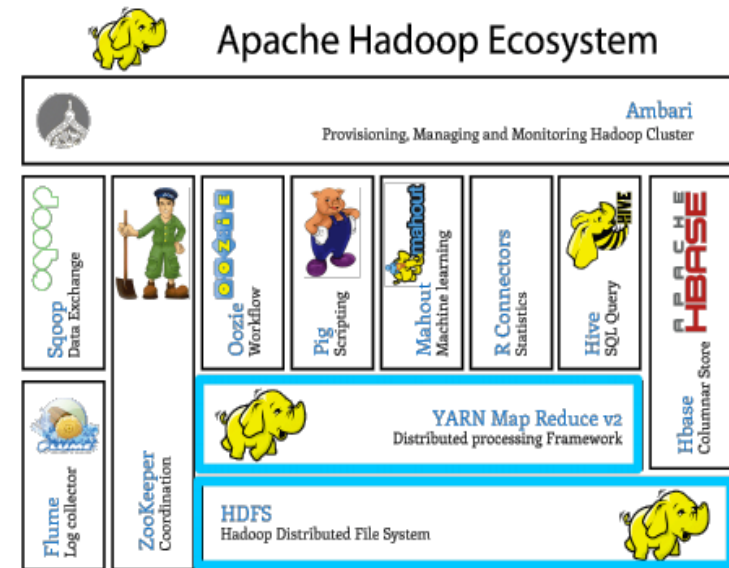
- **Pig** : Langage de script
- **Hive** : Langage proche de SQL (Hive QL)
- **R Connectors** : permet l'accès à HDFS et l'exécution de requêtes Map/Reduce à partir du langage R
- **Mahout** : bibliothèque de machine learning et mathématiques
- **Oozie** : permet d'ordonnancer les jobs Map Reduce, en définissant des workflows



Ecosystème de Hadoop

D'autres outils se situent directement **au dessus de HDFS**, tels que :

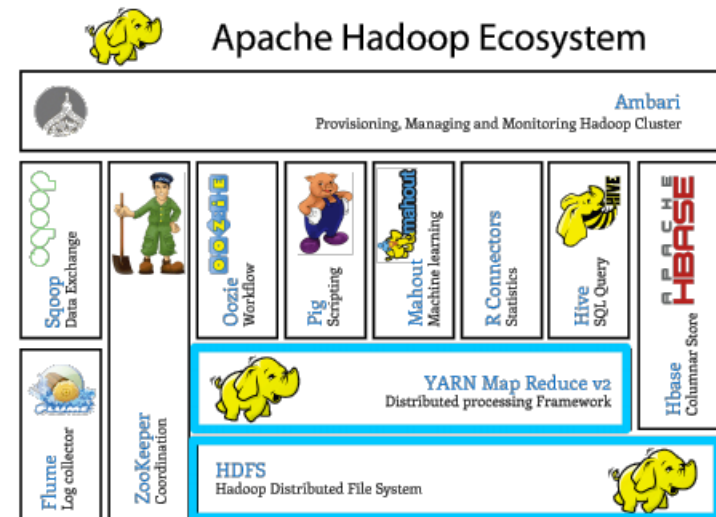
- Hbase : Base de données NoSQL orientée colonnes
- Impala (ne se trouve pas dans la figure) : permet le requêtage de données directement à partir de HDFS (ou de Hbase) en utilisant des requêtes Hive SQL



Ecosystème de Hadoop

Certains outils permettent de **connecter HDFS aux sources externes**, tels que :

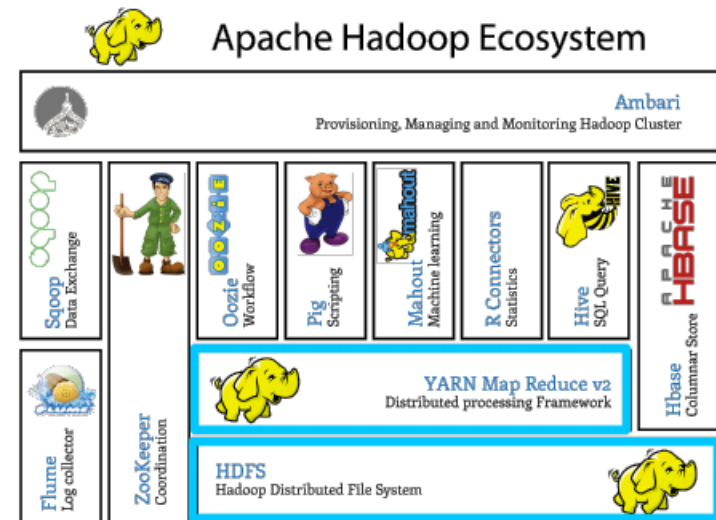
- Sqoop : Lecture et écriture des données à partir de bases de données externes
- Flume : Collecte de logs et stockage dans HDFS



Echystème de Hadoop

Enfin, d'autres outils permettent **la gestion et administration de Hadoop**, tels que :

- **Ambari** : outil pour le provisionnement, la gestion et le monitoring des clusters.
- **Zookeeper** : fournit un service centralisé pour maintenir les informations de configuration, de nommage et de synchronisation distribuée.



HDFS – Hadoop Distributed File System



Datacenter Google



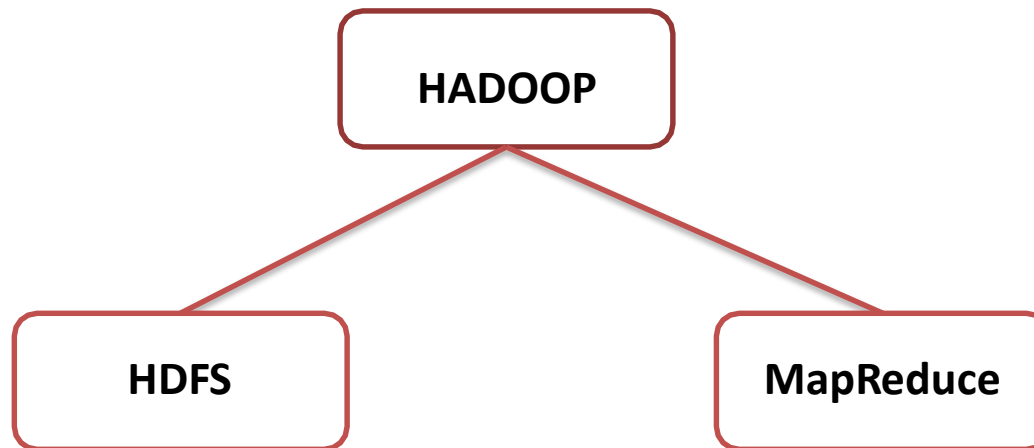
PC Lame

HDFS – Hadoop Distributed File System

- HDFS est un système de fichiers distribué, extensible et portable
- Ecrit en Java
- Permet de stocker de très gros volumes de données sur un Cluster.
- Un cluster est un grand nombre de machines (nœuds) équipées de disques durs et connectées entre elles.

Hadoop: Composants fondamentaux

- Hadoop est constitué de deux grandes parties :
 - Hadoop Distributed File System – **HDFS** : destiné pour le stockage distribué des données
 - Distributed Programming Framework - **MapReduce** : destiné pour le traitement distribué des données.



HDFS : Présentation (1/3)

- **HDFS (Hadoop Distributed File System)** est un système de fichiers :
 - **Distribué**
 - **Extensible**
 - **Portable**
 - Développé par Hadoop, inspiré de GoogleFS et écrit en **Java**.
 - **Tolérant aux pannes**
 - Conçu pour stocker de **très gros volumes** de données sur un grand nombre de machines **peu coûteuses** équipées de disques durs banalisés.
- **HDFS** permet **l'abstraction de l'architecture physique de stockage**, afin de manipuler un système de fichiers distribué comme s'il s'agissait d'un disque dur unique.
- **HDFS reprend de nombreux concepts proposés par des systèmes de fichiers classiques comme ext2 pour Linux ou FAT pour Windows**. Nous retrouvons donc la notion de **blocs** (la plus petite unité que l'unité de stockage peut gérer), les **métadonnées** qui permettent de retrouver les blocs à partir d'un nom de fichier, les **droits** ou encore **l'arborescence** des répertoires.

HDFS : Présentation (2/3)

- **Toutefois, HDFS se démarque d'un système de fichiers classique pour les principales raisons suivantes :**
 - HDFS n'est pas **solidaire du noyau du système d'exploitation**. Il assure une **portabilité** et peut être déployé sur différents systèmes d'exploitation. Un des inconvénients est de devoir solliciter une application externe pour monter une unité de disque HDFS.
 - HDFS est un système **distribué**. Sur un **système classique**, la **taille** du disque est généralement considérée comme la **limite globale** d'utilisation. Dans un système **distribué** comme HDFS, chaque **nœud** d'un cluster correspond à un **sous-ensemble** du volume global de données du cluster. **Pour augmenter ce volume global, il suffira d'ajouter de nouveaux nœuds**. On retrouvera également dans HDFS, un service central appelé Namenode qui aura la tâche de gérer les métadonnées.

HDFS : Présentation (3/3)

- HDFS utilise des **tailles de blocs largement supérieures** à ceux des systèmes classiques. Par défaut, la taille est fixée à **64 Mo**. Il est toutefois possible de monter à 128 Mo, 256 Mo, 512 Mo voire 1 Go. Alors que sur des systèmes classiques, la taille est généralement de **4 Ko**, l'intérêt de fournir des tailles plus grandes **permet de réduire le temps d'accès** à un bloc. Notez que si la taille du fichier est inférieure à la taille d'un bloc, le fichier n'occupera pas la taille totale de ce bloc.
- HDFS fournit un **système de réplication des blocs** dont le nombre de répliques est configurable. Pendant la phase d'écriture, chaque bloc correspondant au fichier est répliqué sur plusieurs nœuds. Pour la phase de lecture, si un bloc est indisponible sur un nœud, des copies de ce bloc seront disponibles sur d'autres nœuds.

HDFS – Hadoop Distributed File System

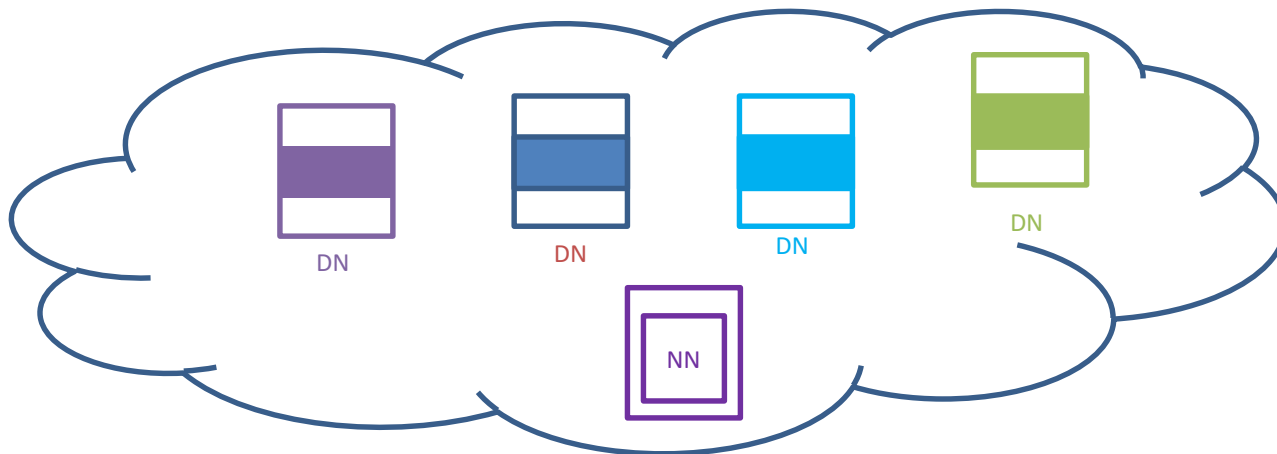
- Quand un fichier `mydata.txt` est enregistré dans HDFS, il est décomposé en grands blocs (par défaut 64Mo), chaque bloc ayant un nom unique : `blk_1`, `blk_2`...
- Les blocs sont numérotés et chaque fichier sait quels blocs il occupe.
- Les blocs d'un même fichier ne sont pas forcément tous sur la même machine.
- La répartition d'un fichier sur plusieurs machines permet d'y accéder simultanément par plusieurs processus.

HDFS – Hadoop Distributed File System

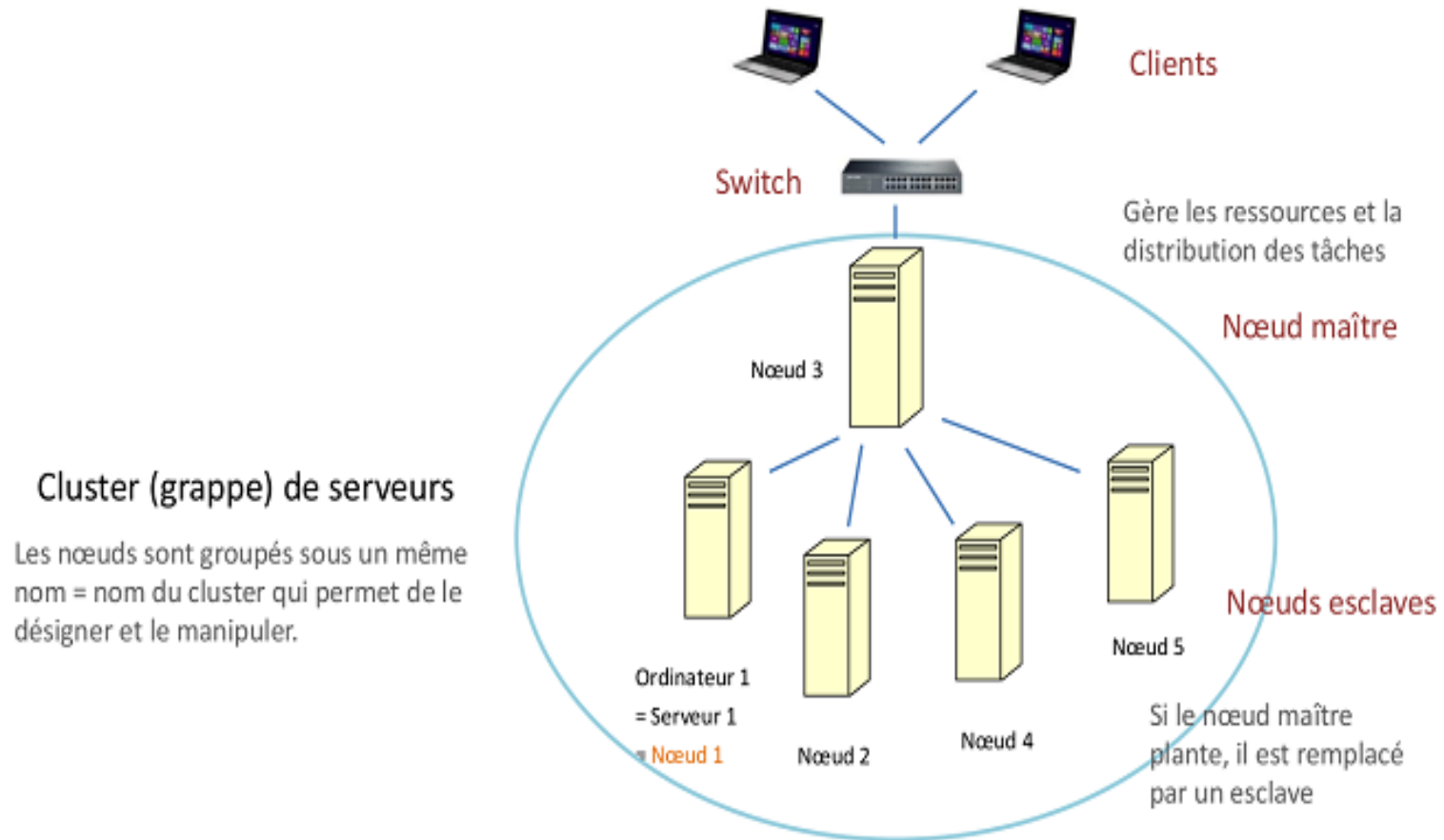
Un cluster HDFS est constitué de machines jouant différents rôles exclusifs entre eux :

Maître HDFS ou namenode : contient tous les noms et blocs des fichiers comme un gros annuaire téléphonique.

Datanodes : toutes les autres machines stockent les blocs du contenu des fichiers.



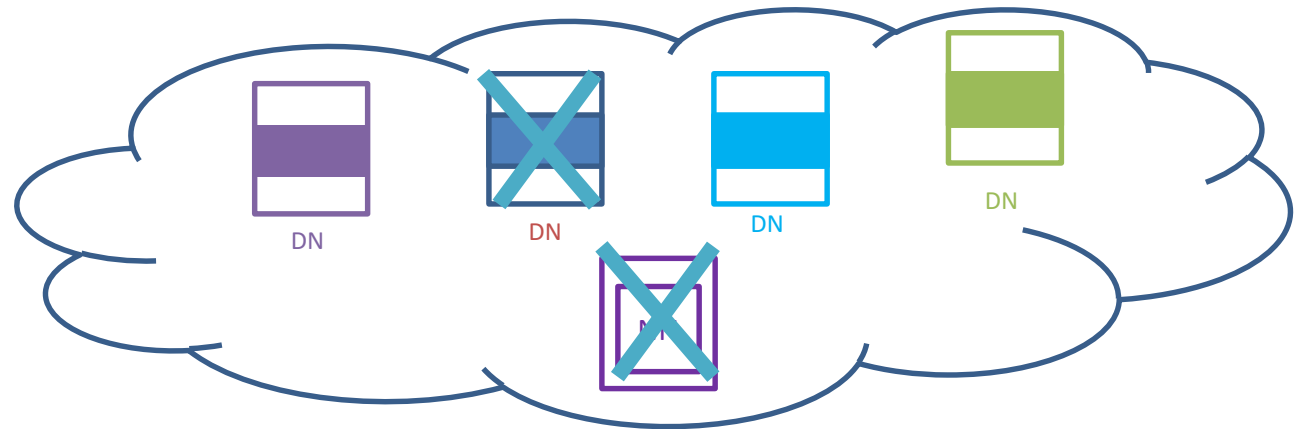
HDFS – Hadoop Distributed File System



HDFS – Hadoop Distributed File System

Problèmes possibles

- Panne d'un **datanode**
- Panne du **namenode**
- Panne réseau

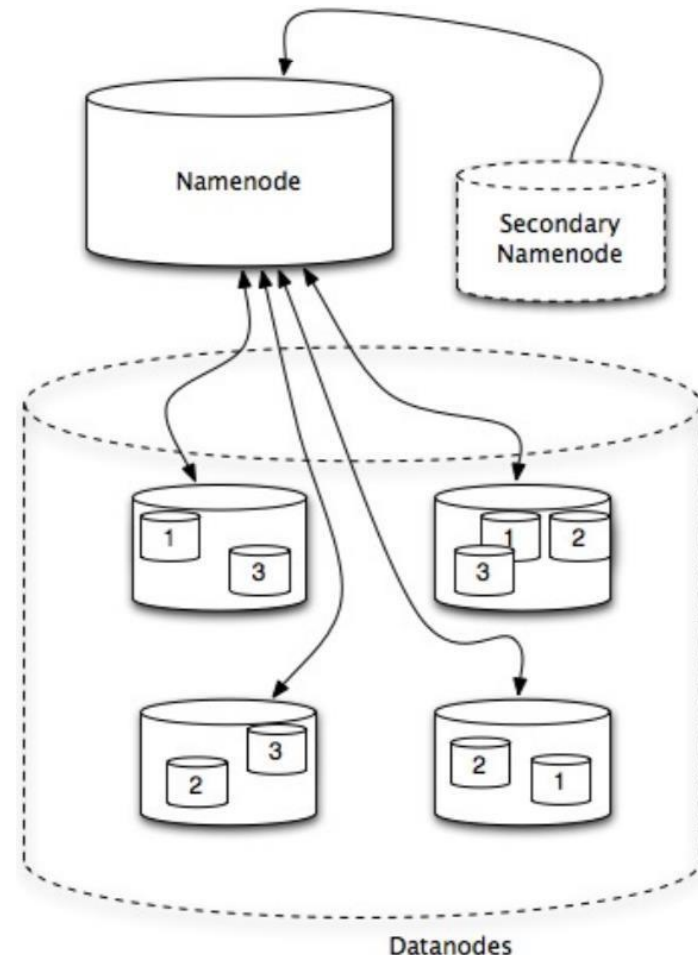


HDFS : Architecture (1/3)

- Une architecture de machines HDFS (aussi appelée cluster HDFS) repose sur trois types de composants majeurs :

— NameNode (noeud de nom) :

Un Namenode est un **service central** (généralement appelé aussi **maître**) qui s'occupe de **gérer l'état** du système de fichiers. Il **maintient l'arborescence** du système de fichiers et **les métadonnées** de l'ensemble des fichiers et répertoires d'un système Hadoop. Le Namenode a une connaissance des Datanodes (étudiés juste après) dans lesquels les blocs sont stockés.



HDFS : Architecture (2/3)

– Secondary Namenode :

Le Namenode dans l'architecture Hadoop est un **point unique de défaillance** (Single Point of Failure en anglais). Si ce service est arrêté, il n'y a pas un moyen de pouvoir extraire les blocs d'un fichier donné. Pour répondre à cette problématique, un Namenode secondaire appelé Secondary Namenode a été mis en place dans l'architecture **Hadoop version2**. Son fonctionnement est relativement simple puisque le **Namenode secondaire vérifie périodiquement l'état du Namenode principal et copie les métadonnées**. Si le **Namenode principal est indisponible, le Namenode secondaire prend sa place**.

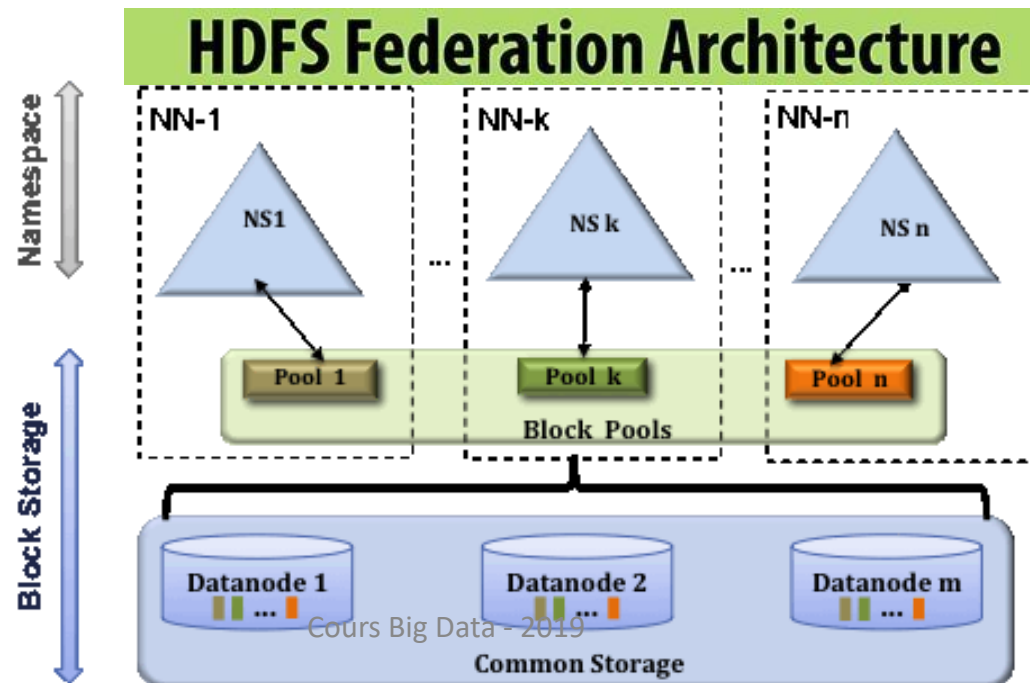
– Datanode :

Précédemment, nous avons vu qu'un Datanode **contient les blocs de données**. En effet, il stocke les blocs de données lui-mêmes. Il y a un DataNode pour chaque machine au sein du cluster. Les Datanodes sont sous les ordres du Namenode et sont surnommés les **Workers**. Ils sont donc sollicités par les Namenodes lors des opérations de lecture et d'écriture.

HDFS : Architecture (3/3)

- Remarque : En plus de l'ajout du Namenode secondaire, nous trouvons d'autres améliorations de HDFS dans la 2^{ème} version de Hadoop :

Les NameNodes étant le point unique pour le stockage et la gestion des métadonnées, ils peuvent être un goulot d'étranglement pour soutenir un grand nombre de fichiers, notamment lorsque ceux-ci sont de petite taille. **En acceptant des espaces de noms multiples desservis par des NameNodes séparés, le HDFS limite ce problème.** Par exemple, un NameNode pourrait gérer tous les fichiers sous /user, et un second NameNode peut gérer les fichiers sous /share.



HDFS – Hadoop Distributed File System

Cas de panne de réseau

➡ Données temporairement inaccessibles si c'est une panne
réseau

HDFS – Hadoop Distributed File System

Cas de panne d'un datanode

➡ Perte de données

Solutions :

- Hadoop réplique chaque bloc **3 fois** (par défaut)
- Il choisit 3 nœuds au hasard, et place une copie du bloc dans chacun d'eux
- Si le nœud est en panne, le **namenode** le détecte, et s'occupe de répliquer encore les blocs qui y étaient hébergés pour avoir toujours **3** copies.
- Concept de Rack Awareness

HDFS – Hadoop Distributed File System

Cas de panne d'un namenode

 Données perdues à jamais si le namenode est défaillant = Mort du HDFS !

Solutions :

- Définition d'un **namenode de secours** appelé **secondary namenode** ;
- Il enregistre des sauvegardes du namenode à intervalles réguliers ;
- Il permet de reprendre le travail si le Namenode actif est défaillant.

HDFS – Hadoop Distributed File System

Le namenode est vital pour HDFS mais unique.

Hadoop 2.x a introduit une nouvelle configuration appelée *high availability* :

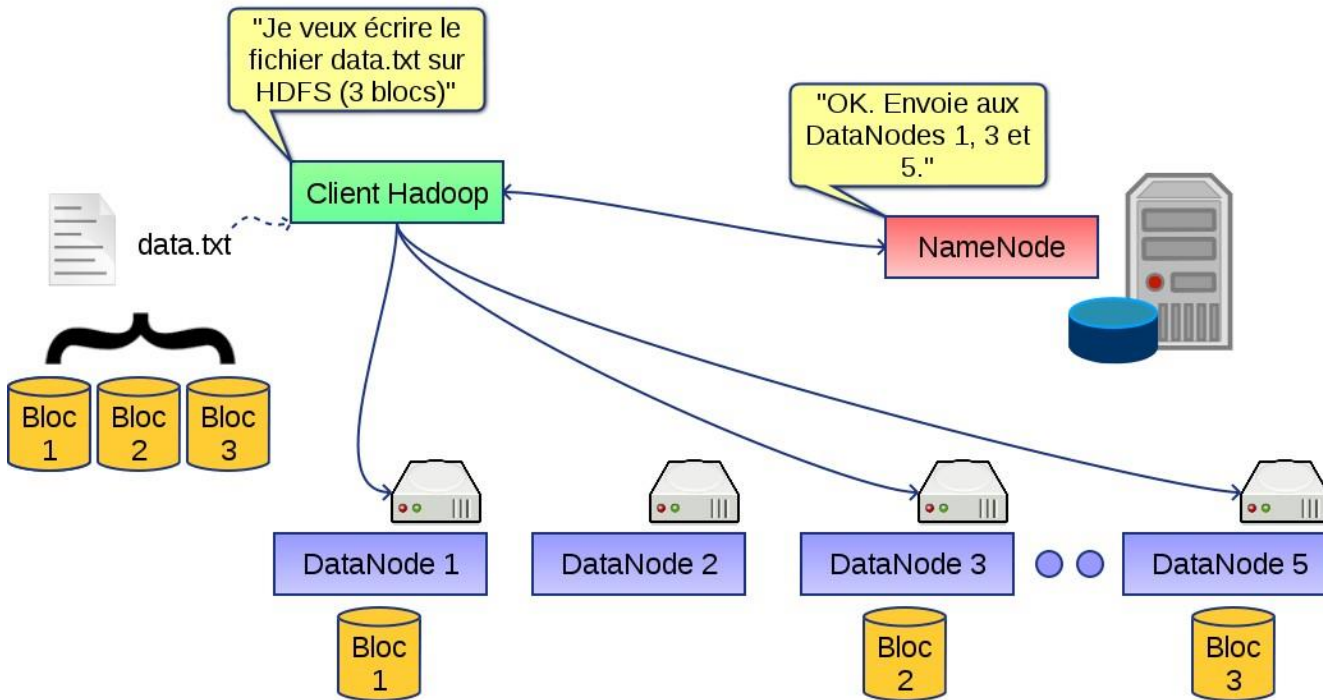
- **2 namenodes de secours** se comportent comme des clones ;
- Ils sont en état d'attente et mis à jour en permanence à l'aide des services appelés *JournalNodes* ;
- Les namenodes de secours sont capables de prendre le relai **instantanément** en cas de panne du namenode ;
- Le secondary namenode fait la même chose que les namenodes de secours ; il devient **alors inutile**.

HDFS: Écriture d'un fichier (1/2)

- Si on souhaite écrire un fichier au sein de HDFS, on va utiliser la commande principale de gestion de Hadoop: **hadoop**, avec l'option **fs**. Mettons qu'on souhaite stocker le fichier `page_livre.txt` sur HDFS.
- Le programme va diviser le fichier en blocs de 64 Mo (ou autre, selon la configuration) – supposons qu'on ait ici 2 blocs. Il va ensuite annoncer au NameNode: « Je souhaite stocker ce fichier au sein de HDFS, sous le nom `page_livre.txt` ».
- Le NameNode va alors indiquer au programme qu'il doit stocker le bloc 1 sur le DataNode numéro 3, et le bloc 2 sur le DataNode numéro 1.
- Le client `hadoop` va alors contacter directement les DataNodes concernés et leur demander de stocker les deux blocs en question. Par ailleurs, les DataNodes s'occuperont – en informant le NameNode – de répliquer les données entre eux pour éviter toute perte de données.

HDFS: Écriture d'un fichier (2/2)

Ecriture HDFS



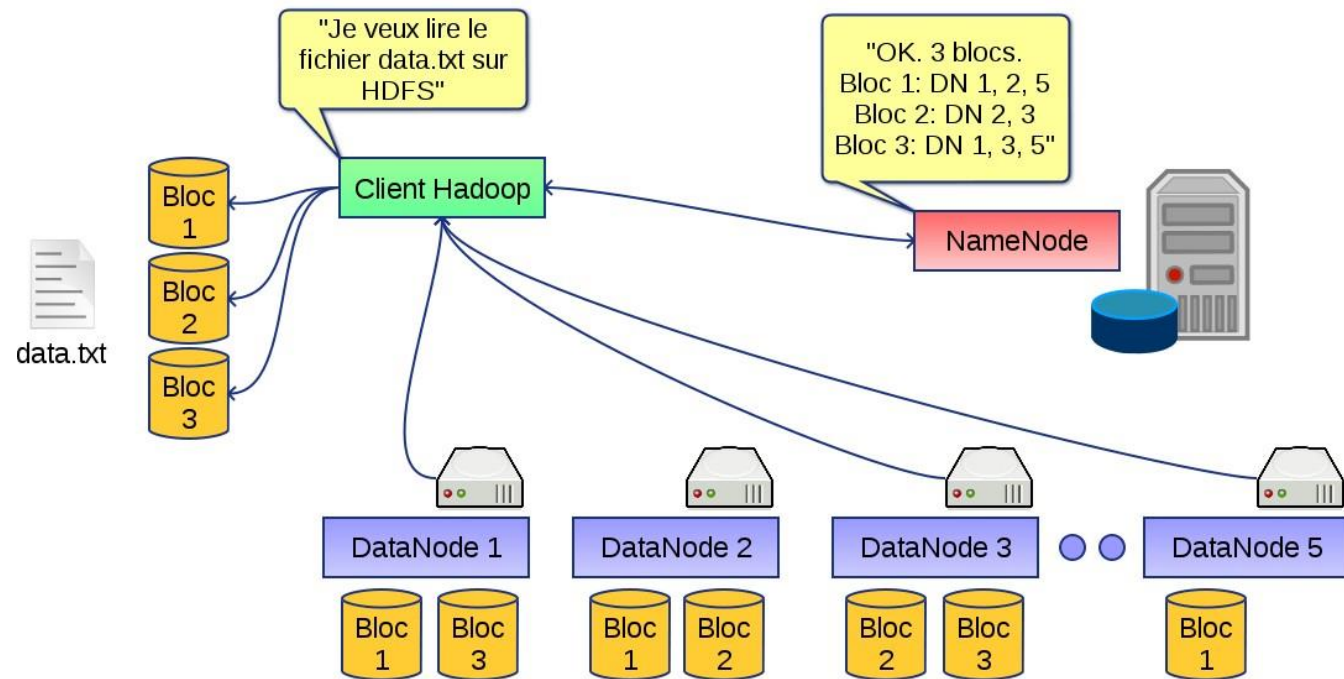
- Le client indique au NameNode qu'il souhaite écrire un bloc.
- Celui-ci lui indique le DataNode à contacter.
- Le client envoie le bloc au Datanode.
- Les DataNodes répliquent le bloc entre eux.
- Le cycle se répète pour le bloc suivant.

HDFS: Lecture d'un fichier (1/2)

- Si on souhaite lire un fichier au sein de HDFS, on utilise là aussi le client Hadoop. Mettons qu'on souhaite lire le fichier `page_livre.txt`.
- Le client va contacter le NameNode, et lui indiquer « Je souhaite lire le fichier `page_livre.txt` ». Le NameNode lui répondra par exemple « Il est composé de deux blocs. Le premier est disponible sur le DataNode 3 et 2, le second sur le DataNode 1 et 3 ».
- Là aussi, le programme contactera les DataNodes directement et leur demandera de lui transmettre les blocs concernés. En cas d'erreur/non réponse d'un des DataNode, il passe au suivant dans la liste fournie par le NameNode.

HDFS: Lecture d'un fichier (2/2)

Lecture HDFS



- Le client indique au NameNode qu'il souhaite lire un fichier.
- Celui-ci lui indique sa taille et les différents DataNode contenant les N blocs.
- Le client récupère chacun des blocs à un des DataNodes.
- Si un DataNode est indisponible, le client le demande à un autre.

HDFS: La commande Hadoop fs

- Comme indiqué plus haut, la commande permettant de stocker ou extraire des fichiers de HDFS est l'utilitaire console `hadoop`, avec l'option `fs`. Il réplique globalement les commandes systèmes standard Linux, et est très simple à utiliser:
- `hadoop fs -put livre.txt /data_input/livre.txt`
 - Pour stocker le fichier `livre.txt` sur HDFS dans le répertoire `/data_input`.
- `hadoop fs -get /data_input/livre.txt livre.txt`
 - Pour obtenir le fichier `/data_input/livre.txt` de HDFS et le stocker dans le fichier local `livre.txt`.
- `hadoop fs -mkdir /data_input`
 - Pour créer le répertoire `/data_input`
- `hadoop fs -rm /data_input/livre.txt`
 - Pour supprimer le fichier `/data_input/livre.txt`
- D'autres commandes usuelles: `-ls`, `-cp`, `-rmr`, `-du`, etc...

Sources

- Cours
 - Ce chapitre est pris essentiellement du cours : Hadoop / Big Data - MBDS - Benjamin Renault (avec quelques modifications).
- Articles
 - « Hadoop et le "Big Data », La Technologie Hadoop au coeur des projets "Big Data" - Stéphane Goumard – Tech day Hadoop, Spark - Arrow-Institute.
 - Comparison Between Hadoop 2.x vs Hadoop 3.x – Data Flair.